

# Redes Neuronales

MLP, funciones de activación y regularización

Sergio Hernández, PhD  
shernandez@ucm.cl



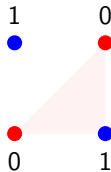
Magíster en Data Science — Universidad Católica del Maule

## Limitaciones de los Modelos Lineales

Los modelos lineales, como la regresión logística o la regresión lineal, son potentes pero tienen una limitación fundamental: solo pueden aprender relaciones lineales entre las características de entrada y la salida.

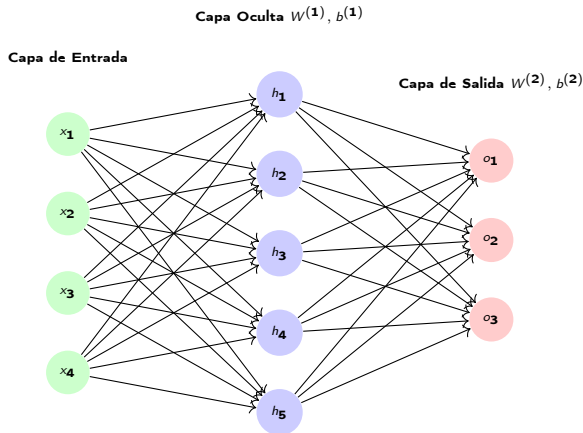
## El problema XOR

Un ejemplo clásico es el problema de la compuerta XOR. No es posible trazar una única línea recta para separar las clases.



# Perceptrón Multicapa

La solución es apilar múltiples capas de neuronas completamente conectadas. A esta arquitectura se le llama comúnmente **Perceptrón Multicapa** (MLP).



Si solo apiláramos transformaciones lineales (afines), la red completa seguiría siendo un modelo lineal.

## No Linealidad

Para aprovechar el potencial de las arquitecturas multicapa, necesitamos incorporar una **función de activación no lineal** ( $\sigma$ ) que se aplica a la salida de cada neurona en las capas ocultas.

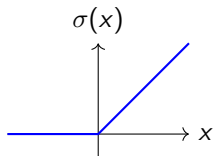
- ▶ La salida de una neurona en la capa oculta incorpora una función de activación no-lineal.

$$h = \sigma(W^{(1)}x + b^{(1)}),$$

$$o = \sigma(W^{(2)}h + b^{(2)}).$$

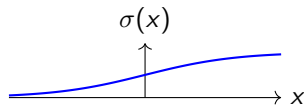
- ▶ Introduce no linealidades que permiten a la red aprender relaciones complejas como la de la compuerta XOR.

## ReLU (Rectified Linear Unit)



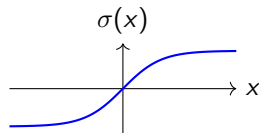
$$\text{ReLU}(x) = \max(x, 0)$$

## Sigmoide



$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

## Tanh (Tangente Hiperbólica)



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

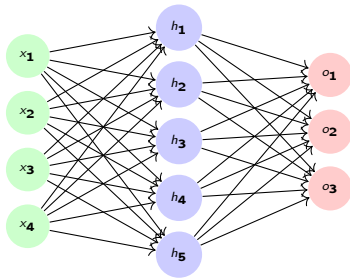
## Retropopagación del error

El objetivo del entrenamiento es encontrar los valores de los parámetros  $\theta = \{W^i, b^i\}_{i=1}^d$  que minimicen una **función de pérdida**  $\mathcal{L}(y, \hat{y})$ .

1. **Propagación hacia adelante (Forward Pass):** Se introduce un lote de datos en la red y se calcula la salida y el error (pérdida).
2. **Propagación hacia atrás (Backward Pass):** Se calcula el gradiente de la función de pérdida con respecto a cada parámetro (pesos  $W^i$  y sesgos  $b^i$ ) de la red, comenzando desde la capa de salida y avanzando hacia atrás.
3. **Actualización de parámetros:**

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

# Backpropagation



$$z = W^{(1)}x + b_1$$

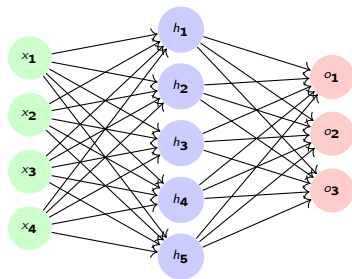
$$h = \sigma(z)$$

$$a = W^{(2)}h + b_2$$

$$o = \sigma(a)$$

$$L = \sum_i \mathcal{L}(y_i, o_i)$$

# Backpropagation



$$\delta^{(2)} = \frac{\partial L}{\partial a} = \frac{\partial L}{\partial o} \odot \sigma'(a)$$

$$\frac{\partial L}{\partial W^{(2)}} = \delta^{(2)} h^T$$

$$\delta^{(1)} = (\partial W^{(2)} \delta^{(2)}) \odot \sigma'(z)$$

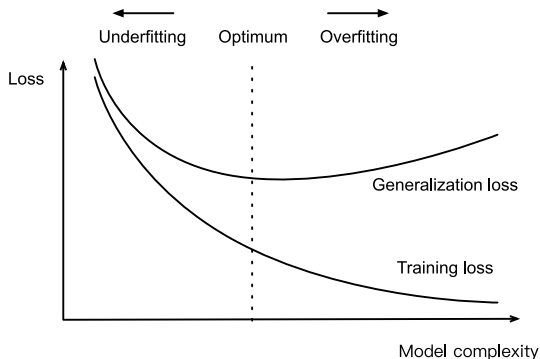
$$\frac{\partial L}{\partial W^{(1)}} = \delta^{(1)} x^T$$

# Sobreajuste (Overfitting)

## Sobreajuste

Las redes neuronales, al tener una gran capacidad (muchos parámetros), corren el riesgo de **sobreajuste**. Ocurre cuando el modelo aprende "demasiado bien" los datos de entrenamiento, incluyendo el ruido, y pierde la capacidad de generalizar a datos nuevos no vistos.

El sobreajuste ocurre cuando una red se apoya excesivamente en un conjunto pequeño de neuronas o características para hacer sus predicciones. Esto la hace vulnerable a pequeñas variaciones en los datos de entrada.

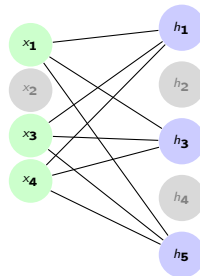
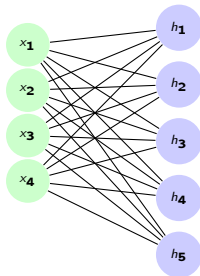


# Técnica de Regularización: Dropout

**Dropout** es una técnica de regularización simple y efectiva introducida para combatir el sobreajuste.

## Dropout

Durante el entrenamiento, en cada paso, se "apagan" (ponen a cero) aleatoriamente un subconjunto de neuronas. Esto obliga a la red a no depender de ninguna neurona individual y a aprender características más redundantes y robustas.



## Mecanismo

Para una capa oculta  $h$ , aplicamos dropout de la siguiente manera:

1. Se define una **probabilidad de abandono**  $p$  (e.g.,  $p = 0.5$ ).
2. Durante el entrenamiento, cada neurona  $h_i$  se pone a cero con probabilidad  $p$ .
3. Las activaciones de las neuronas restantes se escalan por un factor de  $1/(1 - p)$ .

$$h'_i = \begin{cases} 0 & \text{con probabilidad } p \\ \frac{h_i}{1-p} & \text{con probabilidad } 1 - p \end{cases}$$

## Mecanismo

Para una capa oculta  $h$ , aplicamos dropout de la siguiente manera:

1. Se define una **probabilidad de abandono**  $p$  (e.g.,  $p = 0.5$ ).
2. Durante el entrenamiento, cada neurona  $h_i$  se pone a cero con probabilidad  $p$ .
3. Las activaciones de las neuronas restantes se escalan por un factor de  $1/(1 - p)$ .

$$h'_i = \begin{cases} 0 & \text{con probabilidad } p \\ \frac{h_i}{1-p} & \text{con probabilidad } 1 - p \end{cases}$$

## ¿Por qué escalar por $1/(1 - p)$ ?

Esta técnica se llama **Abandono Invertido (Inverted Dropout)**. El escalado asegura que el valor esperado de la salida de la capa se mantenga igual durante el entrenamiento y la prueba.

$$E[h'_i] = p \cdot 0 + (1 - p) \cdot \frac{h_i}{1 - p} = h_i$$

El comportamiento de una capa con Dropout es fundamentalmente diferente en cada fase.

## Durante el Entrenamiento (Training)

- ▶ Dropout está **activo**.
- ▶ En cada iteración de propagación hacia adelante, se genera una máscara de abandono diferente y se aplica a las neuronas.
- ▶ Esto introduce ruido y regulariza el modelo, forzándolo a generalizar.

El comportamiento de una capa con Dropout es fundamentalmente diferente en cada fase.

## Durante el Entrenamiento (Training)

- ▶ Dropout está **activo**.
- ▶ En cada iteración de propagación hacia adelante, se genera una máscara de abandono diferente y se aplica a las neuronas.
- ▶ Esto introduce ruido y regulariza el modelo, forzándolo a generalizar.

## Durante la Inferencia (Test/Predicción)

- ▶ Dropout está **desactivado**.
- ▶ Se utiliza toda la red (todas las neuronas están activas) para hacer la predicción.
- ▶ Esto equivale a promediar las predicciones de la exponencial cantidad de sub-redes generadas durante el entrenamiento.

Entrenar redes profundas puede ser difícil debido a problemas de estabilidad numérica.

$$\partial_{W^{(l)}} o = \underbrace{\partial_{h^{(L-1)}} h^{(L)}}_{M^{(L)} \stackrel{\text{def}}{=}} \cdots \underbrace{\partial_{h^{(l)}} h^{(l+1)}}_{M^{(l+1)} \stackrel{\text{def}}{=}} \underbrace{\partial_{W^{(l)}} h^{(l)}}_{V^{(l)} \stackrel{\text{def}}{=}}. \quad (1)$$

## Desvanecimiento de Gradientes (Vanishing Gradients)

A medida que los gradientes se propagan hacia atrás, pueden volverse extremadamente pequeños, haciendo que las primeras capas de la red aprendan muy lentamente o dejen de aprender. Es común con funciones como la sigmoide.

## Explosión de Gradientes (Exploding Gradients)

Lo contrario: los gradientes se vuelven enormes, provocando actualizaciones muy grandes en los pesos y haciendo que el entrenamiento sea inestable.

- ▶ El Perceptrón Multicapa (MLP) supera las limitaciones de los modelos lineales al introducir **capas ocultas** y **funciones de activación no lineales**.
- ▶ El entrenamiento se realiza mediante el algoritmo de **retropropagación**, ajustando los pesos para minimizar una función de pérdida.
- ▶ El **sobreaajuste** es un riesgo importante, y se combate con técnicas de regularización como **Dropout**.
- ▶ La **estabilidad numérica** (desvanecimiento/explosión de gradientes) es un desafío clave en el entrenamiento, abordado con una correcta inicialización de parámetros y arquitecturas adecuadas.